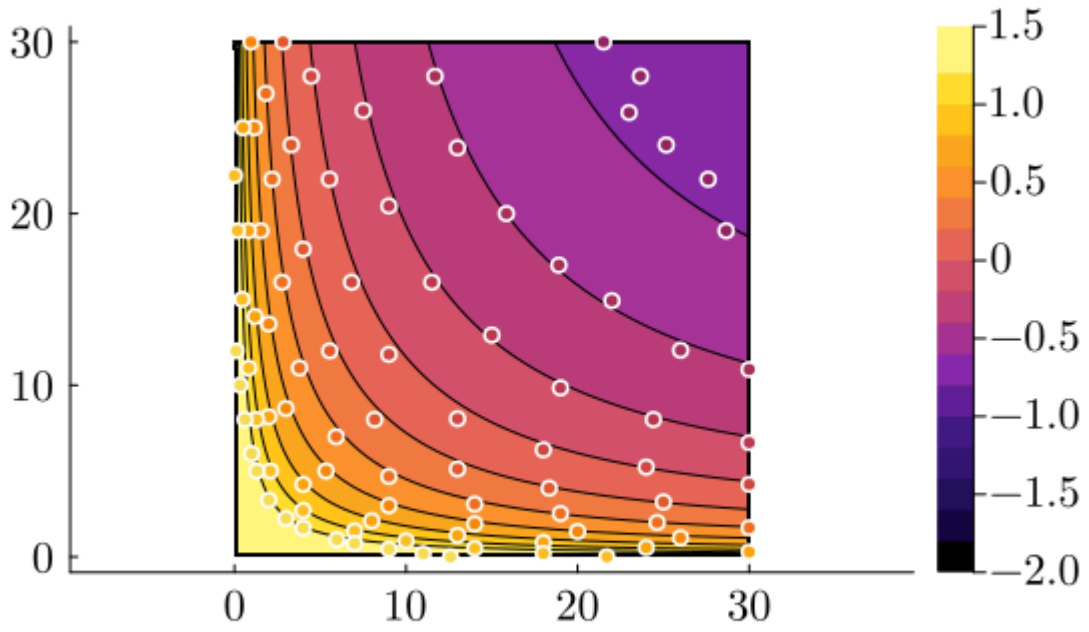



```
begin
    # ======================== PACKAGES ============================
    using GaussianRandomFields
    using CorrelationFunctions
    using CorrelationFunctions.Directional
    using CorrelationFunctions.Utilities: NonPeriodic, Periodic
    using Plots, Measures
    using Statistics
    import Contour as CN
    import Statistics as Stats
    import CorrelationFunctions.Utilities as CFU

    # ======================== PARAMETERS ============================
    # Gaussian random field with correlation G2
    const CORR_LENGTH       = 50.0
    const FIELD_SIZE_X      = 1000
    const FIELD_SIZE_Y      = 1000

    # ε-regularized δ-boundary / level-set at α
    const EPSILON_THICKNESS = 0.02      # ε
    const LEVEL_ALPHA       = 0.1       # α

    # Sampling (ensemble size)
    const N_REALIZATIONS    = 100

    # Shifts for NUMERICAL Fsss^ε
    const MAX_SHIFT_PIXELS  = 30
    const SHIFT_STEP_PIXELS = 1

    # Shifts for ANALYTIC Fsss
    const ANALYTIC_STEP     = 0.1

    # ======================== GAUSSIAN FIELD ========================
    covariance = CovarianceFunction(2, Gaussian(CORR_LENGTH))

```

```
    x_grid = range(1, stop = FIELD_SIZE_X, length = FIELD_SIZE_X)
    y_grid = range(1, stop = FIELD_SIZE_Y, length = FIELD_SIZE_Y)

    gaussian_field = GaussianRandomField(
        covariance,
        CirculantEmbedding(),
        x_grid,
        y_grid;
        minpadding = 1000,
    )

    # ======================== SHIFT VECTORS ==========================

    # integer grid of shifts (triangle legs a, b):
    shift_values_x = 0:SHIFT_STEP_PIXELS:MAX_SHIFT_PIXELS
    shift_values_y = 0:SHIFT_STEP_PIXELS:MAX_SHIFT_PIXELS

    # Required for Fsss: r1=(dx,0,0), r2=(0,dy,0)
    shift_vectors_r1 = [(dx, 0, 0) for dx in shift_values_x, dy in shift_values_y] |> vec
    shift_vectors_r2 = [(0, dy, 0) for dx in shift_values_x, dy in shift_values_y] |> vec

    # Array for storing field realizations Y(r)
    field_realizations = Array{Float64}(undef, FIELD_SIZE_X, FIELD_SIZE_Y, N_REALIZATIONS)

    # ======================== NUMERICAL Fsss^ε (from s3) ==============

    # Ensemble-average of ε-regularized 3-point surface correlation Fsss^ε
    Fsss_eps_avg = zeros(length(shift_values_x), length(shift_values_y))

    for i in 1:N_REALIZATIONS
        # draw sample Y(r)
        field_realizations[:, :, i] = sample(gaussian_field)

        # indicator of 2ε-layer around α  → B_ε(r)
        boundary_eps_2D =
            LEVEL_ALPHA .- EPSILON_THICKNESS .<
            field_realizations[:, :, i] .<
            LEVEL_ALPHA .+ EPSILON_THICKNESS

        # convert to 3D (required by Directional.s3)
        boundary_eps_3D = repeat(boundary_eps_2D, 1, 1, 3)

        # compute 3-point correlation s3 for B_ε(r)  → numerical Fsss^ε
        s3_vals = Directional.s3(
            boundary_eps_3D,
            1,
            shift_vectors_r1,
            shift_vectors_r2;
            mode = Periodic()
        )

        Fsss_eps_avg .+= reshape(
            s3_vals,
            length(shift_values_x),
            length(shift_values_y),
```

```
        )
    end

    Fsss_eps_avg ./= N_REALIZATIONS

    # ======================== ANALYTIC Fsss ==========================

    gaussian_covariance(r) = exp(-(r / CORR_LENGTH)^2)

    """
        analytic_Fsss(r_ac, r_ab, r_bc)

    Exact closed form for the 3-point surface correlation function Fsss
    of the δ-boundary of a truncated Gaussian field.
    """
    function analytic_Fsss(r_ac, r_ab, r_bc)
        G0  = gaussian_covariance(0.0)
        Gab = gaussian_covariance(r_ab)
        Gbc = gaussian_covariance(r_bc)
        Gac = gaussian_covariance(r_ac)

        numer = (Gab + Gac + Gbc) *
                (Gab + Gac + Gbc - 2G0) +
                3G0^2 -
                2Gab^2 - 2Gac^2 - 2Gbc^2

        denom = G0 * (G0^2 - Gbc^2 - Gab^2 - Gac^2) +
                2Gab * Gbc * Gac

        return exp(-LEVEL_ALPHA^2 * numer / denom) / sqrt(abs(denom))
    end

    # ======================== COLOR PALETTE ==========================
    custom_colors = [
        "#000004", "#1a0a47", "#291060", "#381778",
        "#471a88", "#6f229f", "#972faf",
        "#b13881", "#cb4973", "#e35c5f", "#f07547",
        "#f98e32", "#fda31f", "#fec41c",
        "#ffdb29", "#fff67d"
    ]

    # ======================== ANALYTIC GRID ==========================

    analytic_x = 0:ANALYTIC_STEP:MAX_SHIFT_PIXELS
    analytic_y = 0:ANALYTIC_STEP:MAX_SHIFT_PIXELS

    analytic_Fsss_matrix =
        [analytic_Fsss(hypot(x, y), x, y) for y in analytic_y, x in analytic_x] ./
(2pi^(3/2))

    log_analytic = log10.(analytic_Fsss_matrix)

    # ======================== NUMERICAL GRID =========================

    # Normalizing Fsss^ε
    Fsss_eps_numeric =
        2 .* Fsss_eps_avg ./ (EPSILON_THICKNESS^3) ./ (2pi^(3/2))

```

```
    log_numeric = log10.(Fsss_eps_numeric)

    ny, nx = size(log_numeric)
    numeric_x = 0:SHIFT_STEP_PIXELS:(nx-1)
    numeric_y = 0:SHIFT_STEP_PIXELS:(ny-1)

    # ======================== LEVELS & CONTOURS =====================

    finite_vals = filter(isfinite, log_analytic[:])
    log_min, log_max = Stats.quantile!(finite_vals, (0.01, 0.99))

    plot_levels = collect(-2.0:0.2:1.5)

    numeric_contours = CN.contours(
        numeric_x,
        numeric_y,
        log_numeric,
        plot_levels[1:end-1]
    )

    # ======================== PLOT ==============================

    Plots.default(fontfamily = "Computer Modern",
                  guidefontsize = 14,
                  tickfontsize = 14,
                  legendfontsize = 14,
                  titlefontsize = 14)

    plt = contourf(
        analytic_x,
        analytic_y,
        log_analytic;
        levels = plot_levels,
        fill = true,
        c = cgrad(custom_colors),
        clims = (-2.0, 1.5),
        colorbar = true,
        aspect_ratio = :equal,
        grid = :off,
        legend = :outertopright,
    )

    xs = Float64[]
    ys = Float64[]
    zs = Float64[]

    for lvl in CN.levels(numeric_contours)
        v = CN.level(lvl)
        for line in CN.lines(lvl)
            xline, yline = CN.coordinates(line)
            npts = length(xline)

            rel = (v - log_min) / (log_max - log_min + eps())
            step = max(1, Int(round(npts / (6 + 7*rel))))

            idx = 1:step:npts
            append!(xs, xline[idx])
            append!(ys, yline[idx])
```

```
            append!(zs, fill(v, length(idx)))
        end
    end

    scatter!(
        plt,
        xs,
        ys;
        marker_z = zs,
        markersize = 4,
        markerstrokewidth = 2,
        markerstrokecolor = :white,
        c = :black,
        label = false,
    )

    display(plt)
```

```
8332812 negative eigenvalues ≥ -9.094947017729282e-13 detected, Gaussian random
field will be
        approximated (ignoring all negative eigenvalues); increase padding if p
ossible

GR: highest contour level less than maximal z value is not supported.
```